

Préparation au Concours National Commun — Informatique MP

Épreuve d'Informatique

Filière MP — Durée : 4 heures

*Structures de données et algorithmes de la blockchain BITCOIN*Auteur : **Pr Agrégé El Hadiq Zouhair****cpgeacademy.org** — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit.

Pour aller plus loin. Les élèves peuvent traiter une **nouvelle version** de ce problème dotée d'un **autocorrecteur des réponses**, et suivre un **cours complet sur Bitcoin**, sur **cpgeacademy.org**.

Consignes. L'usage de toute calculatrice et de tout document est interdit. Toutes les fonctions demandées seront écrites en **Python 3**. On pourra librement réutiliser les fonctions écrites dans les questions précédentes. La clarté, la rigueur des justifications et la qualité de la rédaction seront prises en compte. Le sujet comporte cinq parties *largement indépendantes*, de difficulté croissante.

Présentation et notations

BITCOIN est un système de monnaie électronique décentralisé. Ce problème étudie quelques structures de données et algorithmes qui en constituent le cœur : la représentation des montants, l'encodage des données, les arbres de MERKLE, la preuve de travail, et la sélection des fonds à dépenser.

On adopte les notations suivantes, valables dans tout le sujet.

- Les montants sont comptés en **satoshis**, la plus petite unité ; un bitcoin (noté BTC) vaut exactement 10^8 satoshis.
- Pour un entier $a \geq 0$ et $k \in \mathbb{N}$, $a \gg k$ désigne le décalage binaire à droite, c'est-à-dire $\lfloor a/2^k \rfloor$.
- $\log_2$ désigne le logarithme en base 2 ; $\lfloor x \rfloor$ (resp. $\lceil x \rceil$) la partie entière inférieure (resp. supérieure).
- En Python, les entiers sont de taille arbitraire ; on considérera qu'une opération arithmétique élémentaire (+, −, comparaison, $\gg$) sur des entiers bornés par une constante du problème s'effectue en temps $O(1)$.
- On dispose d'une fonction de hachage $h : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ (le double SHA-256), supposée *résistante aux collisions* : il est calculatoirement impossible d'exhiber deux entrées distinctes $x \neq y$ telles que $h(x) = h(y)$. En Python, `h(b)` renvoie un objet `bytes` de 32 octets, et `b1 + b2` désigne la concaténation de deux `bytes`.

Partie I — Émission monétaire (arithmétique des montants)

La création de bitcoins suit une règle déterministe. La *subvention* attribuée au mineur d'un bloc de hauteur $h \in \mathbb{N}$ vaut 50 BTC pour les premiers blocs, puis elle est divisée par deux (*halving*) tous les 210 000 blocs.

Question 1. Écrire une fonction `vers_satoshis(btc)` qui, étant donné un montant `btc` (entier ou flottant), renvoie le nombre *entier* de satoshis correspondant, arrondi au plus proche. On rappelle que `round` réalise l'arrondi.

Question 2. Expliquer en une ou deux phrases pourquoi les logiciels du réseau manipulent les montants en satoshis entiers plutôt qu'en bitcoins flottants.

Question 3. On note $s_0 = 5\,000\,000\,000$ le nombre de satoshis correspondant à 50 BTC. La subvention d'un bloc de hauteur h vaut, en satoshis,

$$\sigma(h) = \begin{cases} s_0 \gg \lfloor h/210000 \rfloor & \text{si } \lfloor h/210000 \rfloor < 64, \\ 0 & \text{sinon.} \end{cases}$$

Écrire une fonction `subvention(h)` qui calcule $\sigma(h)$ en utilisant l'opérateur de décalage $\gg$.

Question 4. Déterminer la plus petite hauteur h^* à partir de laquelle la subvention est nulle. Justifier.

Question 5. Écrire une fonction `emission_totale()` qui renvoie le nombre total de satoshis jamais créés, c'est-à-dire

$$\Sigma = \sum_{k=0}^{+\infty} 210000 \times (s_0 \gg k).$$

On expliquera pourquoi cette somme ne comporte en réalité qu'un nombre *fini* de termes non nuls, et on précisera la complexité temporelle de la fonction.

Question 6. Démontrer l'inégalité $\Sigma \leq 21\,000\,000 \times 10^8$. (On pourra majorer $s_0 \gg k$ par $s_0/2^k$ et sommer la série géométrique.) En déduire qu'il n'existera jamais plus de 21 millions de bitcoins.

Partie II — Encodage Base58 et entiers CompactSize

Pour écrire de manière compacte et lisible de grands entiers (adresses, clés), BITCOIN utilise un encodage en base 58. On dispose de la chaîne des 58 symboles :

ALPHABET = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"

Le symbole d'indice r ($0 \leq r \leq 57$) est ALPHABET[r].

Question 7. Écrire une fonction `base58(n)` qui, pour un entier $n \geq 1$, renvoie la chaîne de caractères représentant n en base 58 (chiffre de poids fort en tête). On procédera par divisions euclidiennes successives par 58.

Question 8. On note $n \geq 1$. Montrer que la boucle de `base58(n)` effectue exactement $\lfloor \log_{58} n \rfloor + 1$ tours. En déduire la complexité en nombre de divisions.

Question 9. Justifier la terminaison de `base58` en exhibant un **variant de boucle** (une quantité entière positive strictement décroissante).

Les compteurs et longueurs (nombre d'entrées, d'octets, etc.) sont encodés au format COMPACTSIZE. Un entier n tel que $0 \leq n < 2^{64}$ est représenté par une suite d'octets $b_0 b_1 \dots$ selon la règle :

$$\begin{array}{ll} 0 \leq n \leq 252 & \text{un seul octet égal à } n ; \\ 253 \leq n \leq 2^{16} - 1 & \text{l'octet } 0xfd \text{ suivi de } n \text{ sur 2 octets, } \textit{petit-boutiste} ; \\ 2^{16} \leq n \leq 2^{32} - 1 & \text{l'octet } 0xfe \text{ suivi de } n \text{ sur 4 octets, } \textit{petit-boutiste} ; \\ 2^{32} \leq n \leq 2^{64} - 1 & \text{l'octet } 0xff \text{ suivi de } n \text{ sur 8 octets, } \textit{petit-boutiste}. \end{array}$$

Petit-boutiste signifie que l'octet de poids faible est écrit en premier. En Python, si `d` est un objet `bytes`, alors `d[0]` est un entier, et `int.from_bytes(d, "little")` interprète `d` comme un entier petit-boutiste ; réciproquement `n.to_bytes(k, "little")` produit les `k` octets de `n`.

Question 10. Écrire une fonction `lire_varint(d)` qui, étant donné un objet `bytes d` commençant par un entier COMPACTSIZE, renvoie la valeur de cet entier.

Question 11. Écrire une fonction `ecrire_varint(n)` qui renvoie l'encodage `COMPACTSIZE` de l'entier n ($0 \leq n < 2^{64}$) sous forme d'un objet `bytes`, en choisissant toujours la représentation la plus courte.

Question 12. Démontrer que pour tout entier n tel que $0 \leq n < 2^{64}$, on a `lire_varint(ecrire_varint(n)) == n`. (*On distinguera les quatre cas de la table.*)

Partie III — Arbres de Merkle

Un bloc résume ses transactions par un unique condensé, la **racine de Merkle**, calculée ainsi. On part de la liste des condensés des feuilles $[f_0, f_1, \dots, f_{n-1}]$ (avec $n \geq 1$, chaque f_i étant un objet `bytes`). Tant qu'il reste plus d'un élément : si le nombre d'éléments est impair, on *duplique* le dernier ; puis on remplace la liste par la liste des condensés $h(f_{2i} || f_{2i+1})$ des paires consécutives. Le dernier élément restant est la racine.

Question 13. Écrire une fonction `racine_merkle(feuilles)` qui prend la liste non vide `feuilles` et renvoie la racine (un objet `bytes`). Le cas $n = 1$ renverra f_0 .

Question 14. Soit $n \geq 1$ le nombre de feuilles. Montrer que le nombre total d'appels à h effectués par `racine_merkle` est au plus $2n$. (*On pourra majorer, à chaque niveau, le nombre de paires par la moitié de la taille courante augmentée de 1, puis sommer.*) En déduire que la complexité est $O(n)$ appels à h .

Question 15. On appelle **preuve d'inclusion** de la feuille d'indice i la liste des condensés « voisins » rencontrés sur le chemin de f_i jusqu'à la racine, chacun accompagné de sa position (à gauche ou à droite du condensé courant). Écrire une fonction `preuve(feuilles, i)` qui renvoie cette liste sous forme d'une liste de couples (`voisin`, `gauche`) où `voisin` est un objet `bytes` et `gauche` un booléen valant `True` si le voisin se trouve à gauche.

Question 16. Écrire une fonction `verifier(feuille, preuve, racine)` qui, en repliant la feuille le long de la preuve, renvoie `True` si et seulement si le condensé reconstruit est égal à `racine`. (*Si `gauche` vaut `True`, le condensé courant c devient $h(\text{voisin} || c)$, sinon $h(c || \text{voisin})$.*)

Question 17. On suppose h résistante aux collisions. Montrer que si `verifier(f, p, r)` renvoie `True` pour la racine authentique r d'un arbre, alors, sauf à exhiber une collision de h , la feuille f figurait effectivement parmi les feuilles de cet arbre. (*On raisonnera par récurrence sur la longueur de la preuve.*)

Question 18. Montrer que pour un bloc de n transactions, la preuve d'inclusion d'une feuille comporte $O(\log n)$ condensés. Commenter l'intérêt de cette propriété pour un client léger (téléphone) qui ne stocke pas toute la chaîne.

Partie IV — Preuve de travail (minage)

Miner un bloc consiste à trouver une valeur, le **nonce**, telle que le condensé de l'en-tête, interprété comme un entier, soit inférieur ou égal à une **cible** T (un entier tel que $0 < T < 2^{256}$). On modélise l'en-tête par une fonction `entete(nonce)` renvoyant un objet `bytes`, et on note $H(x) = \text{int.from_bytes}(h(x), \text{"big"})$ l'entier associé au condensé.

Question 19. Écrire une fonction `valide(nonce, T)` qui renvoie `True` si et seulement si $H(\text{entete}(\text{nonce})) \leq T$.

Question 20. Écrire une fonction `miner(T)` qui renvoie le plus petit `nonce` entier ≥ 0 tel que `valide(nonce, T)` soit vrai.

Question 21. On modélise le hachage comme idéal : pour chaque `nonce`, l'entier $H(\text{entete}(\text{nonce}))$ est une variable aléatoire *uniforme* sur $\{0, 1, \dots, 2^{256} - 1\}$, indépendante d'un nonce à l'autre. On pose $p = \frac{T + 1}{2^{256}}$.

- (a) Justifier que la probabilité qu'un nonce donné soit valide vaut p .
 (b) Soit N le nombre de nonces essayés par `miner(T)` jusqu'au premier succès inclus. Reconnaître la loi de N et donner son espérance $\mathbb{E}[N]$.

Question 22. Montrer que `miner(T)` termine *presque sûrement* : la probabilité que les k premiers essais échouent tous tend vers 0 quand $k \rightarrow +\infty$. Que vaut cette probabilité en fonction de p et k ?

La cible T est stockée dans l'en-tête sous une forme compacte de 4 octets appelée `nBits`. Si `bits` est l'entier correspondant, on pose

$$\text{exposant} = \text{bits} \gg 24, \quad \text{mantisse} = \text{bits} \& 0\text{ffffff}, \quad T = \text{mantisse} \times 256^{\text{exposant}-3},$$

où $\&$ désigne le *et* binaire.

Question 23. Écrire une fonction `bits_vers_cible(bits)` qui calcule T d'après cette formule.

Question 24. Toutes les 2016 blocs, la cible est réajustée pour viser un intervalle moyen de 10 minutes par bloc. Si l'ancienne cible est T et que les 2016 derniers blocs ont mis un temps réel `reel` secondes pour un temps visé `visé` secondes, la nouvelle cible est

$$T' = T \times \frac{\text{clamp}(\text{reel})}{\text{visé}}, \quad \text{clamp}(x) = \min\left(\max\left(x, \frac{\text{visé}}{4}\right), 4\text{visé}\right).$$

Écrire une fonction `reajuster(T, reel, vise)` renvoyant $\lfloor T' \rfloor$ (on utilisera la division entière `//`).
 Démontrer ensuite l'encadrement $\frac{T}{4} \leq T' \leq 4T$.

Partie V — Sélection des fonds à dépenser (programmation dynamique)

Pour payer un montant S (en satoshis, $S \geq 1$), un portefeuille doit choisir un sous-ensemble de ses *sorties non dépensées* (UTXO) dont la somme atteint au moins S . On dispose d'une liste d'entiers strictement positifs $v = [v_0, \dots, v_{n-1}]$ (les montants des UTXO). On souhaite choisir $I \subseteq \{0, \dots, n-1\}$ minimisant $(\sum_{i \in I} v_i) - S$ sous la contrainte $\sum_{i \in I} v_i \geq S$ (autrement dit minimiser la monnaie rendue). On note $C = \sum_i v_i$ la somme totale disponible.

Question 25. Le problème de décision suivant est-il, à votre avis, « facile » ? *Étant donné v et un entier S , existe-t-il I tel que $\sum_{i \in I} v_i = S$ exactement ? Citer le problème classique auquel il se ramène et sa classe de complexité. (Aucune démonstration n'est demandée, seulement une identification.)*

Question 26. On se ramène à énumérer les sommes *atteignables*. Pour $0 \leq s \leq C$, on note $A(s)$ le prédicat « il existe un sous-ensemble de v de somme exactement s ». Donner une relation de récurrence permettant de calculer les $A(s)$ en traitant les v_i un par un, et préciser l'initialisation.

Question 27. En déduire et écrire une fonction `atteignables(v, C)` qui renvoie une liste de booléens `acc` de longueur $C + 1$ telle que `acc[s]` vaut `True` si et seulement si s est atteignable. (*Attention à l'ordre de parcours des sommes pour ne pas réutiliser un même v_i deux fois.*)

Question 28. Écrire une fonction `meilleure_selection(v, S)` qui renvoie le plus petit montant atteignable $\geq S$ (donc le montant total optimal à engager), ou `-1` s'il n'en existe aucun.

Question 29. Donner, en la justifiant, la complexité temporelle et spatiale de `atteignables` en fonction de n et C . Pourquoi qualifie-t-on cet algorithme de *pseudo-polynomial* ?

Question 30. Démontrer la correction de la récurrence de la Question précédente : montrer, par récurrence sur le nombre k d'éléments déjà traités, l'invariant « `acc[s]` vaut `True` si et seulement s'il existe un sous-ensemble des k premiers éléments de v de somme s ». Conclure sur la correction de `meilleure_selection`.

Question 31. Justifier la terminaison de `atteignables` et de `meilleure_selection`.

— Fin de l'énoncé —